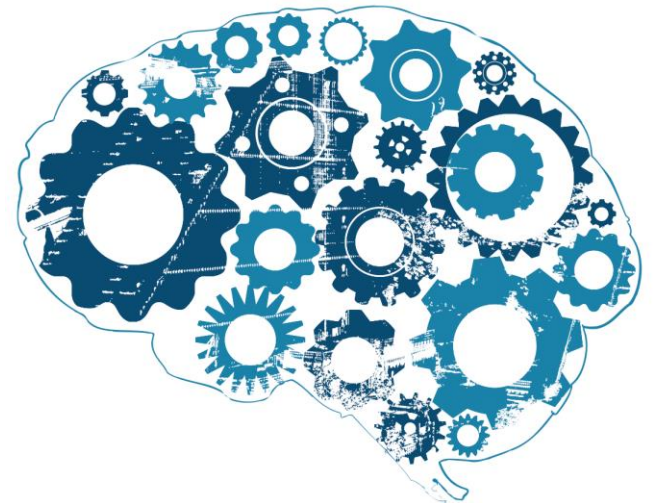


Infrastructure as Code

DevOps Practices

Ron C. Chiang

University of St. Thomas

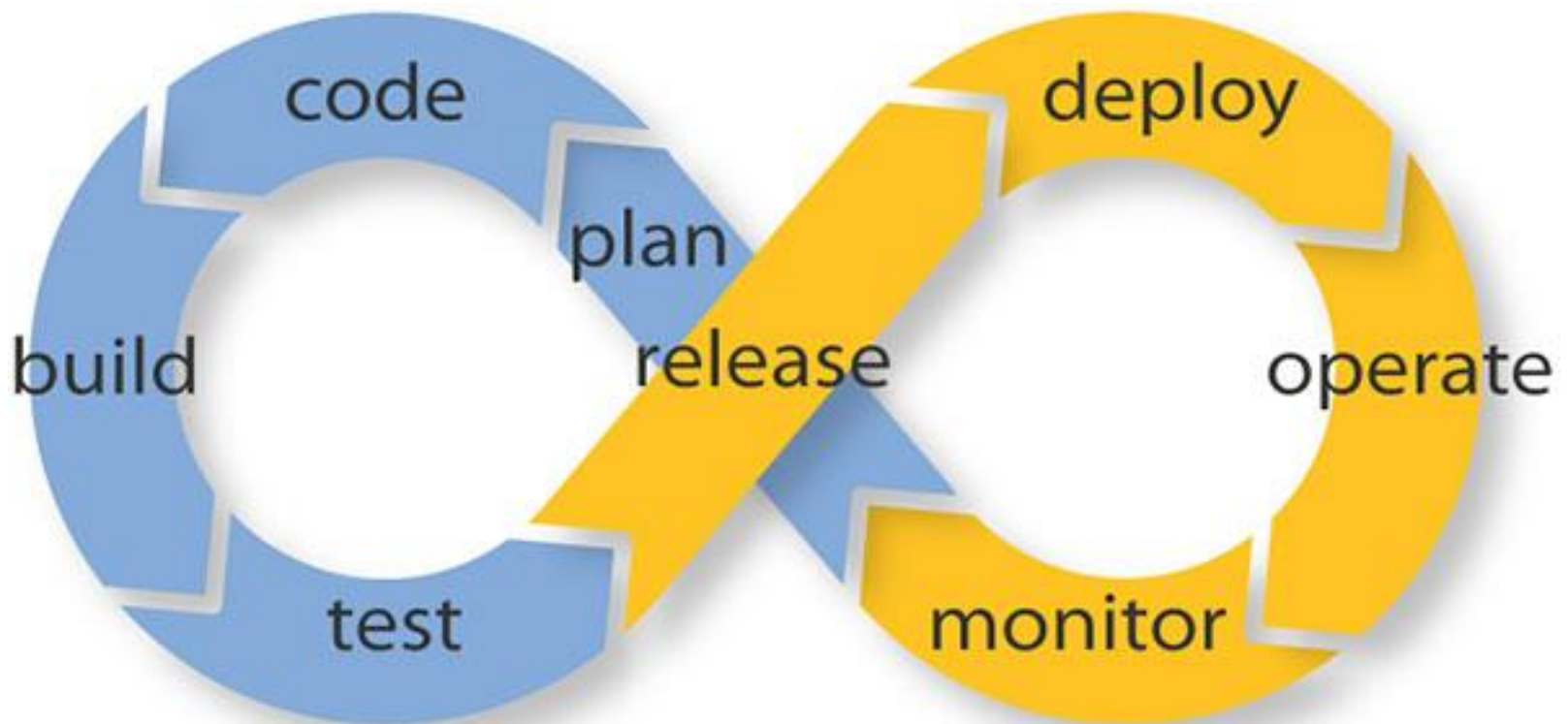


Agenda

- DevOps
- Lean theory
- DevOps Culture
- Design For Operations
- Infrastructure as code
- Code Pipeline

DevOps

A **collaborative** effort to improve the **quality** and **velocity** of the service lifecycle.



Is DevOps really new?

- Operations staff have been automating system configurations for decades
 - Shell scripts
 - PXE boots
 - VMWare templates
- Developers have also been automating software releases as well.
 - Make, Ant, SVN...
- DevOps builds upon decades of practices, but we still have challenges...

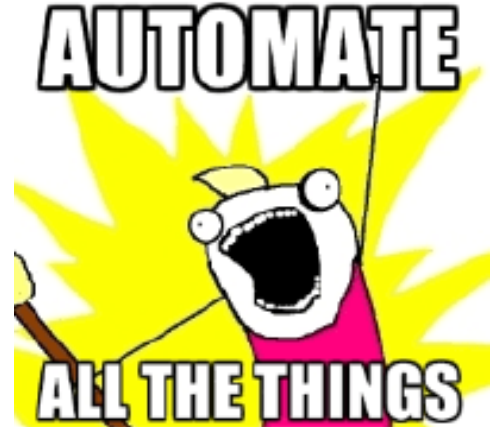
Common DevOps Misconceptions

“We have to hire special DevOps people and build a DevOps team.”



Common DevOps Misconceptions

“DevOps is just about automating all the things.”



- DevOps is mostly about culture and building a shared responsibility for service delivery.

Common DevOps Misconceptions

“We use DevOps tools therefore we must be *doing* DevOps.”

- Vendors can sell you tools, but you cannot buy culture.



DevOps CALMS Framework



C – Culture

A – Automation

L – Lean

M – Measurement

S – Sharing

DevOps culture and sharing



1. Weeks writing code
2. Weeks (**Testing**) code
3. Hand it over the **OPS** team for deployment



Development



Wall of Confusion



Operations

**WORKED FINE IN
DEV**

OPS PROBLEM NOW

Conflicting measures of success

- Development success is measured by the rate of change during a release cycle -- lines of code, code commits, team velocity.
- Operations success is measured by service uptime, performance, and resiliency..

**IT'S
NOT
FAIR!**



Cultural priorities

- **Collaborative**
 - Teams are multi-disciplined and focused on service delivery (the greater good).
- **Transparent**
 - People share information with one another.
 - Blame-less environment
- **Change oriented**
 - Change is expected and embraced.

Cultural priorities

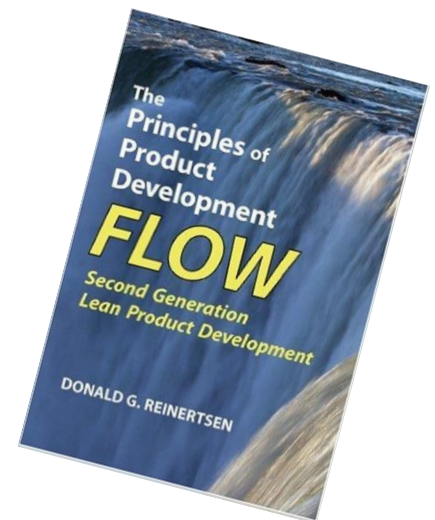


Does my organization have a DevOps culture?

- Do our developers and operations team members sit near one another and go to lunch together?
- Do operations people regularly attend inception, showcase, and retrospective development meetings?
 - And do developers go to operations standups?
- Do developers understand in detail how their code is deployed?

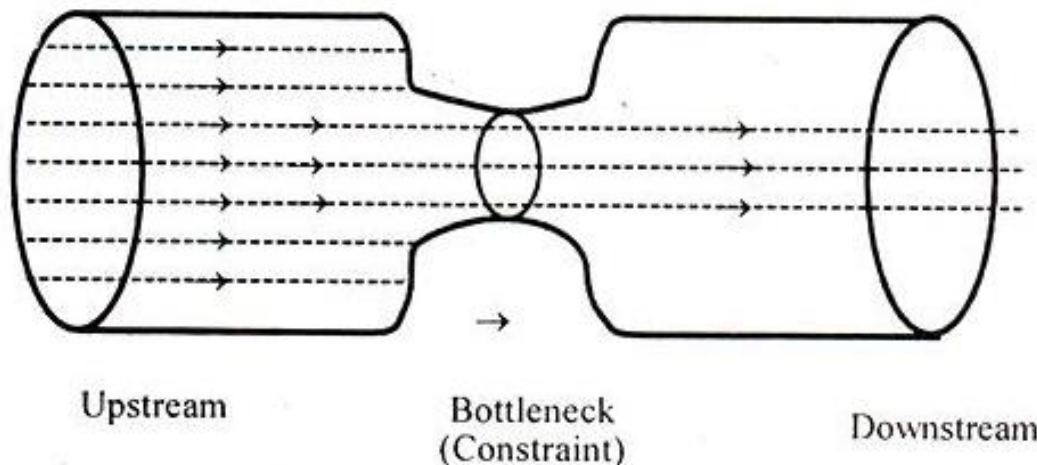
DevOps Culture & Lean

- DevOps culture is heavily influenced by **Lean** practices.
- Lean is a wide-spread methodology in manufacturing.
 - Commonly implemented as Six Sigma practices
 - Goals:
 - Eliminate waste within a system (“Muda”)
 - Improve product quality
 - Respect for people
 - Continuous improvement (“Kaizen”)



Theory of Constraints

- Developed by Eli Goldratt:
 - Think of your process as a chain of links.
 - Improving strong links doesn't strengthen the chain.
 - Improve the weakest link first.



Theory of Constraints

- Example: If it requires 40 **hours** of effort to test a release and 10 **minutes** to deploy it, then the testing process is a constraint on improving cycle time.



Lean Queuing Theory

- What happens when we push work through a complex system?
- **Traditional utilization thinking:** focus on operating resources at peak capacity.
 - Optimize locally rather than at the system level.
- **Throughput thinking:** focus on identifying the constraints that reduce delivery effectiveness.

Lean Queuing Theory

- Software development is a **stochastic system** with queues (non-linear and non-deterministic).
 - Behavior of this type of system defies our instincts.
 - Large projects (batch sizes) increase cycle time.
 - Take longer to process leading to variability and more queues
 - Increasing worker utilization (just work harder) makes it worse!!
- **Little's Law:** The only way to maximize throughput is by minimizing Work In Progress (WIP) through the system using small batch sizes.
 - Example: Small Agile sprints or Kanban tasks

Design

Developer-land

Identify
Business
Process

Match
Existing
Functions
and Start
Modeling

Build and
Compose

Service Lifecycle

Evaluate

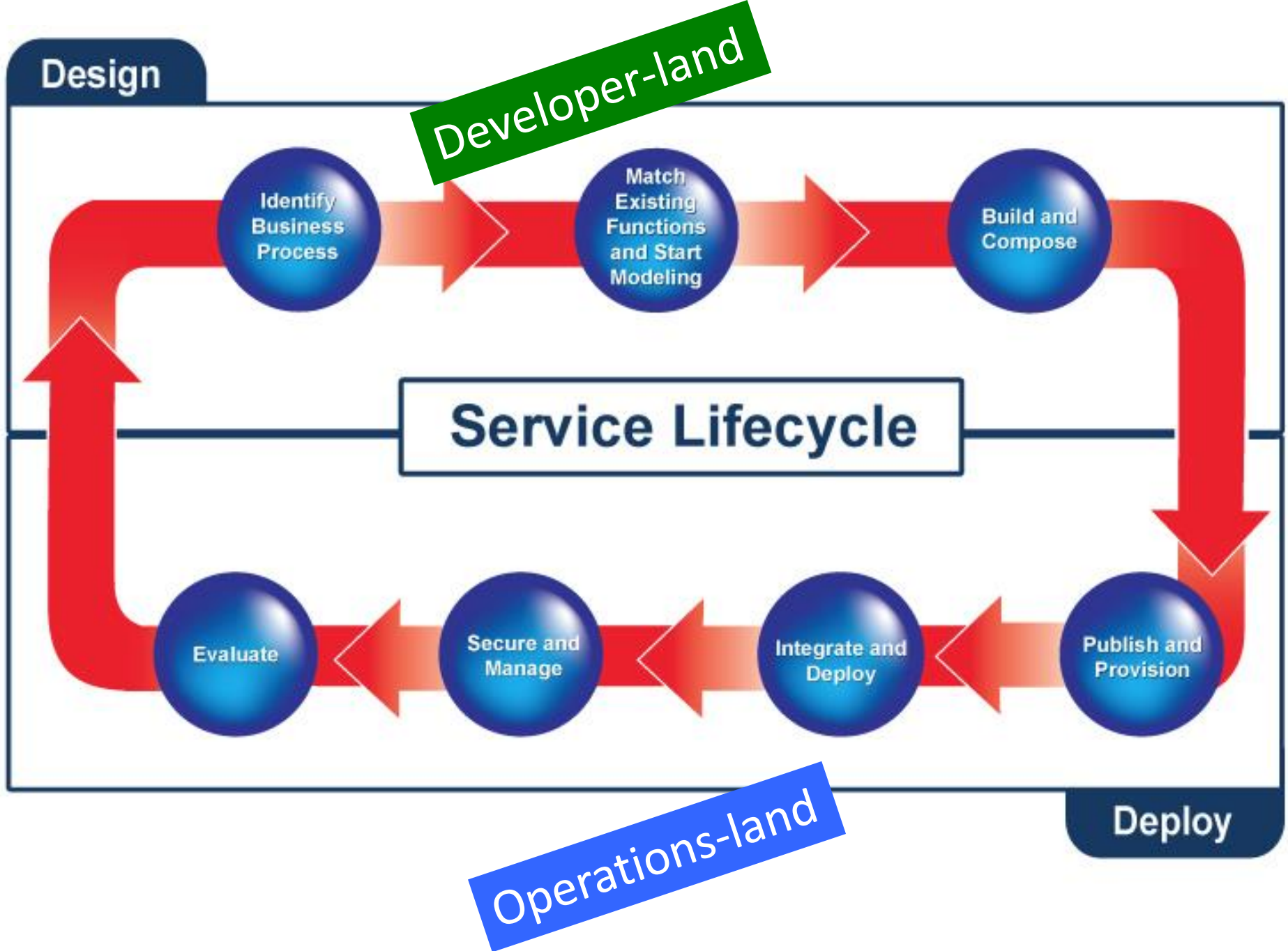
Secure and
Manage

Integrate and
Deploy

Publish and
Provision

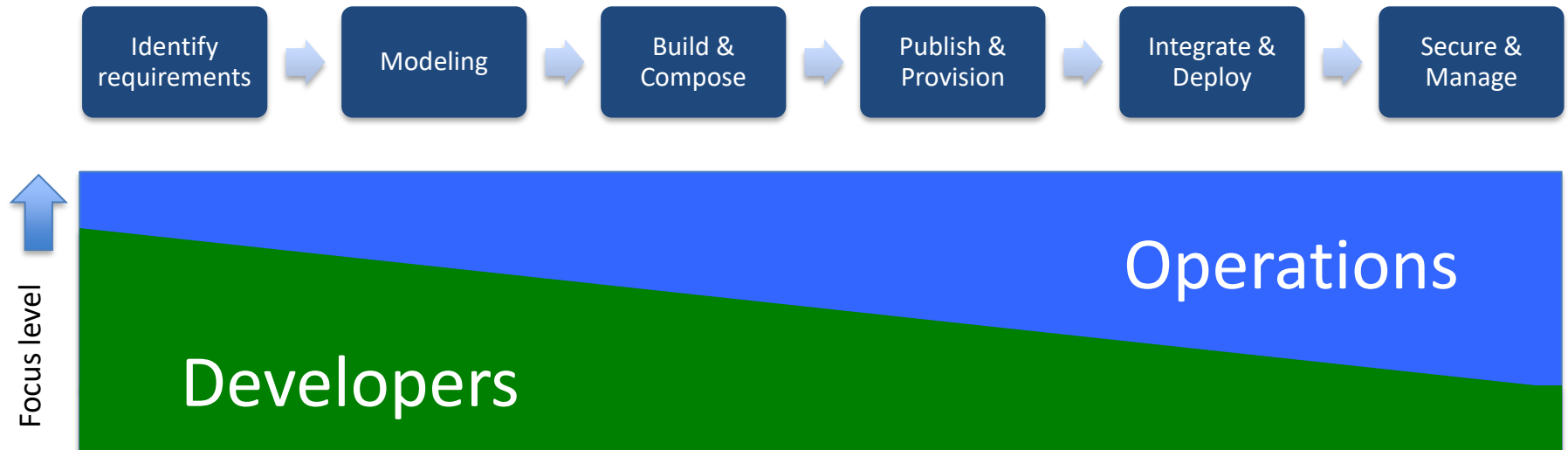
Deploy

Operations-land



DevOps Organization Focus

- Developers more focused on earlier phases of lifecycle.
- Operations more focused on later phases of lifecycle.
- **Shared accountability** for service delivery.



Design For Operations

- Problem: Software designs oftentimes don't account for deployment, management, and support phases.
- Non-functional requirements:
 - How will we scale this feature?
 - How will we provide high availability?
 - How will our customer service team support this?
- In a DevOps culture, operations staff participate in software planning discussions.
- Operations ensures non-functional requirements are part of the software design.
 - Availability, Scalability, Upgradeability, Supportability, Testability, etc.

Developers role

- Developers participate in service delivery and support.
 - Builds accountability: “Eat your own dog food”.
 - Developers are part of the on-call support team.
- If Developers provide service delivery support, do they get root?
 - Not necessarily, but they should have enough access to troubleshoot common issues.
 - Full access to all infrastructure and application logs.
 - Full access to metrics.
 - Ability to remediate issues by pushing out corrective updates.

Transparency

- In DevOps organizations, staff are empowered to improve quality.
 - Lean principle: anyone can stop the production line at any time.
 - Everyone works together to solve the quality problem.
 - It's the highest priority job.
- Service issues are addressed at postmortem meetings.
 - Scheduled no later than 48 hours after incidents.
 - Focus on determining root cause, not assigning blame,
 - Corrective action given the highest priority.

Transparency

- A transparent, blameless environment is a safer environment for employees.
 - More willing to participate in discussions and raise awareness of potential problems.
 - Employees won't hide mistakes.
 - More willing to experiment and test ways to improve the service.
- Survey statistics show that employees are happier in this environment.

Embrace Change

- DevOps cultures recognize that change is inevitable, so they embrace it.
- Lean/Agile: Smaller batch sizes = increasing rate of change.
- The more we change, the better we get at it.
 - Sports teams can't get together every 3 months and expect to win a championship. They have to practice.
- Learn to change often:
 - Monthly -> Weekly -> Daily -> Hourly
 - Should be able to push most changes at any time of day without impacting services.
 - Amazon & Netflix deploy thousands of times a day.

Embrace Change

- But some changes can't happen in the middle of the day like database schema changes, right?
 - Many database changes do not require a service maintenance window if designed and sequenced carefully.
 - Zero Downtime Deployment strategy
 - Change process may require multiple software updates to transform the schema from one version to another.
 - Evolutionary Database Design
 - This process is hard for traditional organizations because their cycle time is too long.
 - It's easier for them to just disrupt the service and push out the changes.

Automation



Automation

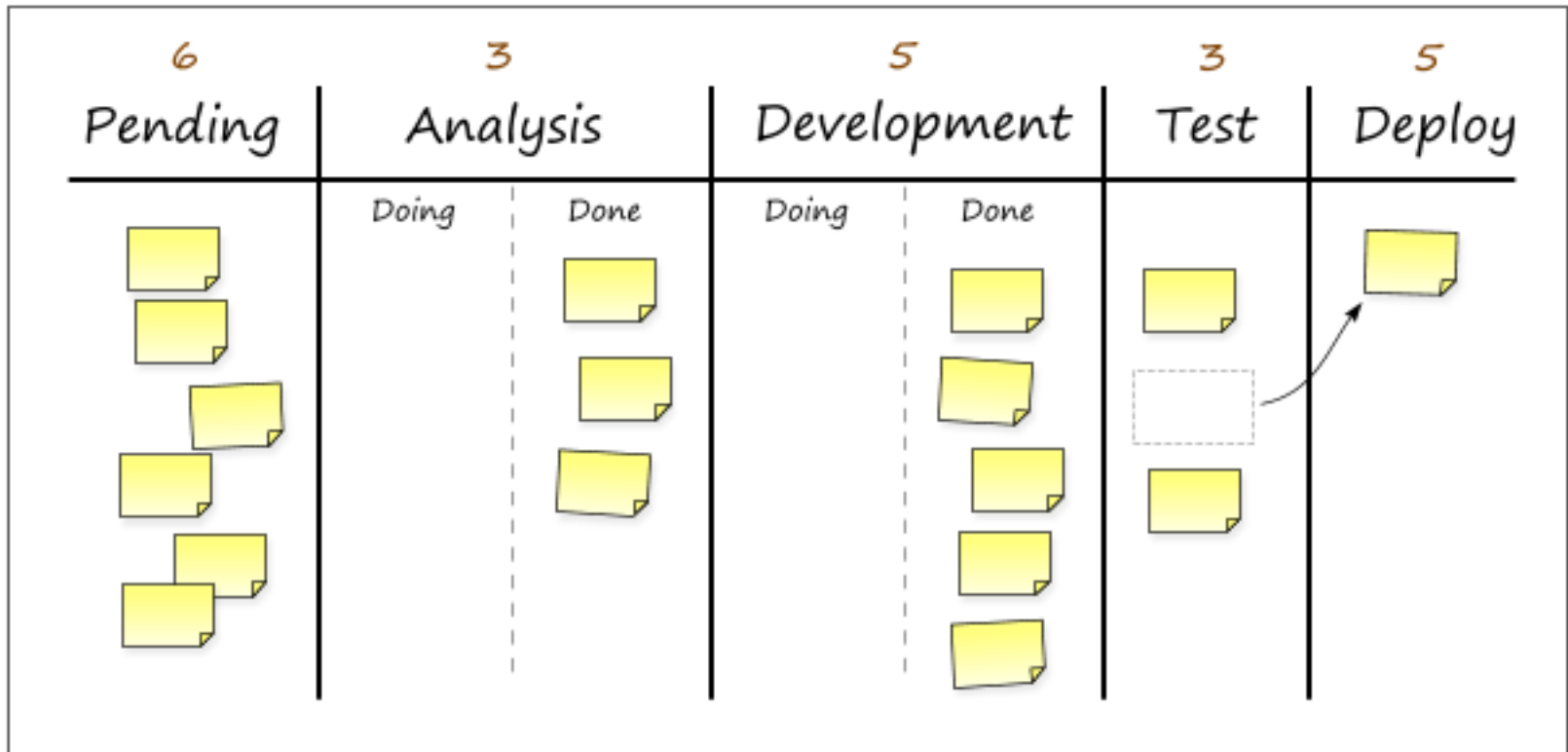
- DevOps mantra: Automate everything.
 - Humans are brittle.
 - Processes must be repeatable.
 - Best way to reduce cycle time and improve quality.
 - Most cost effective way to scale work.

Software Assembly Line

- DevOps treats software development like an automated manufacturing line.
 - Design the product
 - Assemble it
 - Test it
 - Package it for deployment
- Work moves through the company, queuing up at various stations.
 - For example: product backlog and QA tests are very visible sorts of queues.

Software Assembly Line

Example: Kanban board showing queues



Code Pipeline

- A code pipeline is an automated process for getting software from version control into the hands of your users.
- Key benefits to this pattern:
 - Effectively prevents the deployment of code that has not been thoroughly tested.
 - Development and release process becomes rapid, repeatable, and reliable.
 - Releases become normal events and can occur much more frequently.

Code pipeline enables continuous change flow.



Continuous Integration

- Continuous integration is a process used in the first stage of the code pipeline.
 - Originated as part of Extreme Programming (XP) practices.
 - Traditionally, developers would merge code changes once a day and do a nightly build.
 - Don't break the build!
 - Developers would work on separate long-running feature branches and face “merge hell” when trying to merge branches months later.

Continuous Integration Practices

- Developers check-in code at least once a day to the same mainline (master) branch.
- Code is built and unit tested at every check-in.
- Everyone can view the build and test reports.
- Code committed to the mainline branch may be deployed to production.
- Deliverables are stored in an artifact repository and deployed to other test environments after a successful build.



Software testing automation

- Committed code is automatically tested in a series of stages providing greater assurance of working software.
- Oftentimes companies focus their efforts on discovering defects versus preventing defects.
 - Manual exploratory testing
 - End-to-end testing
 - User Acceptance testing

Software testing automation

- DevOps adopts lessons from Lean.



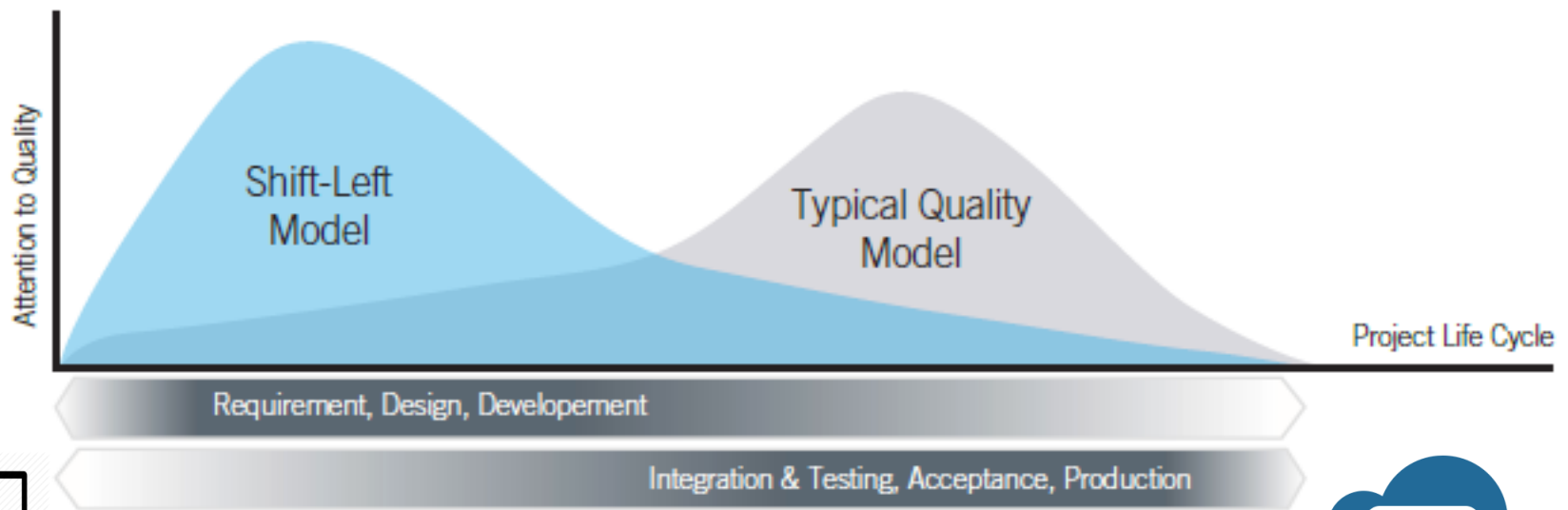
Cease dependence on inspection to achieve quality. Eliminate the need for inspection on a mass basis by building quality into the product in the first place.

W. Edwards Deming

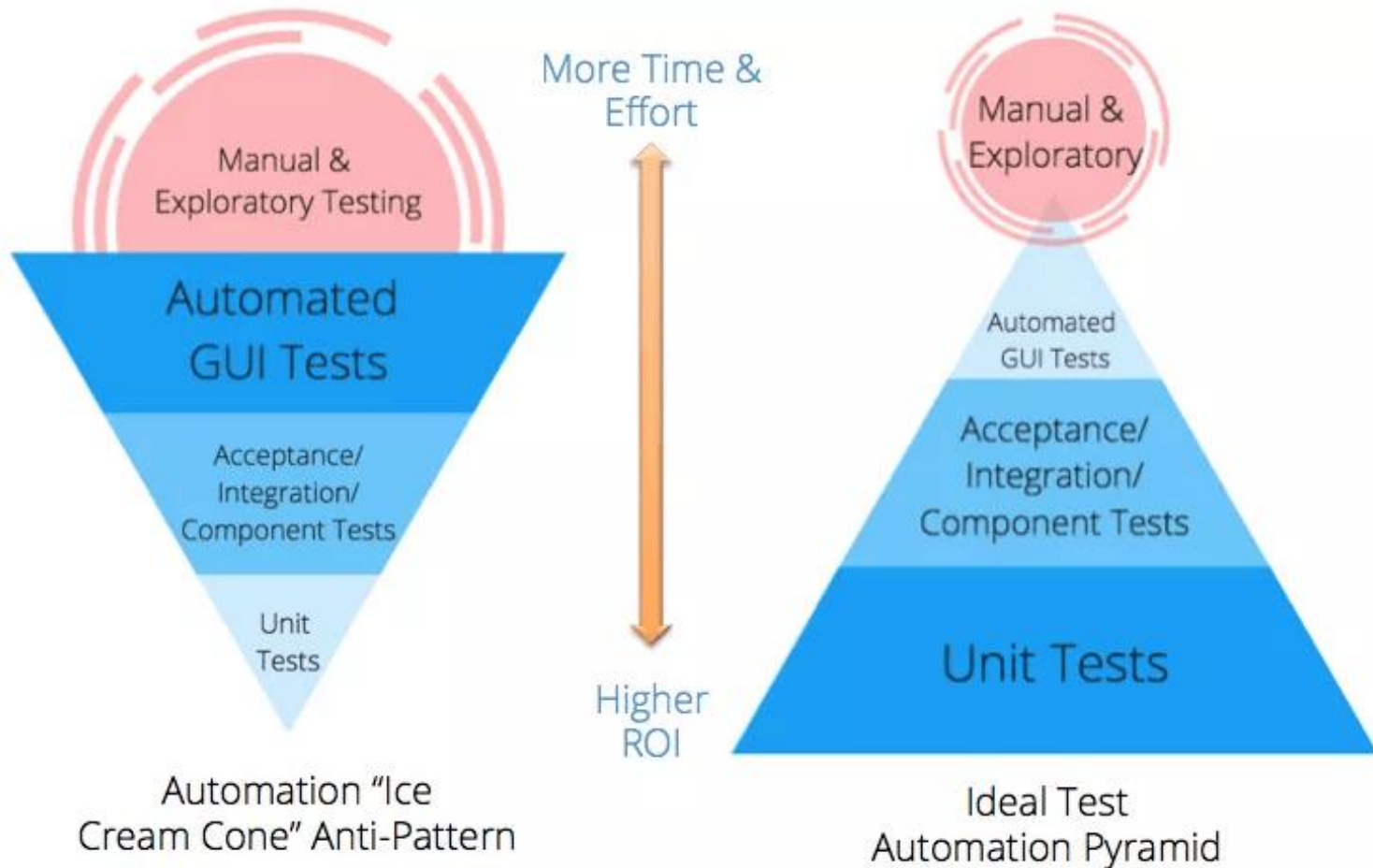
source: quotes.deming.org/1611

Shift Left

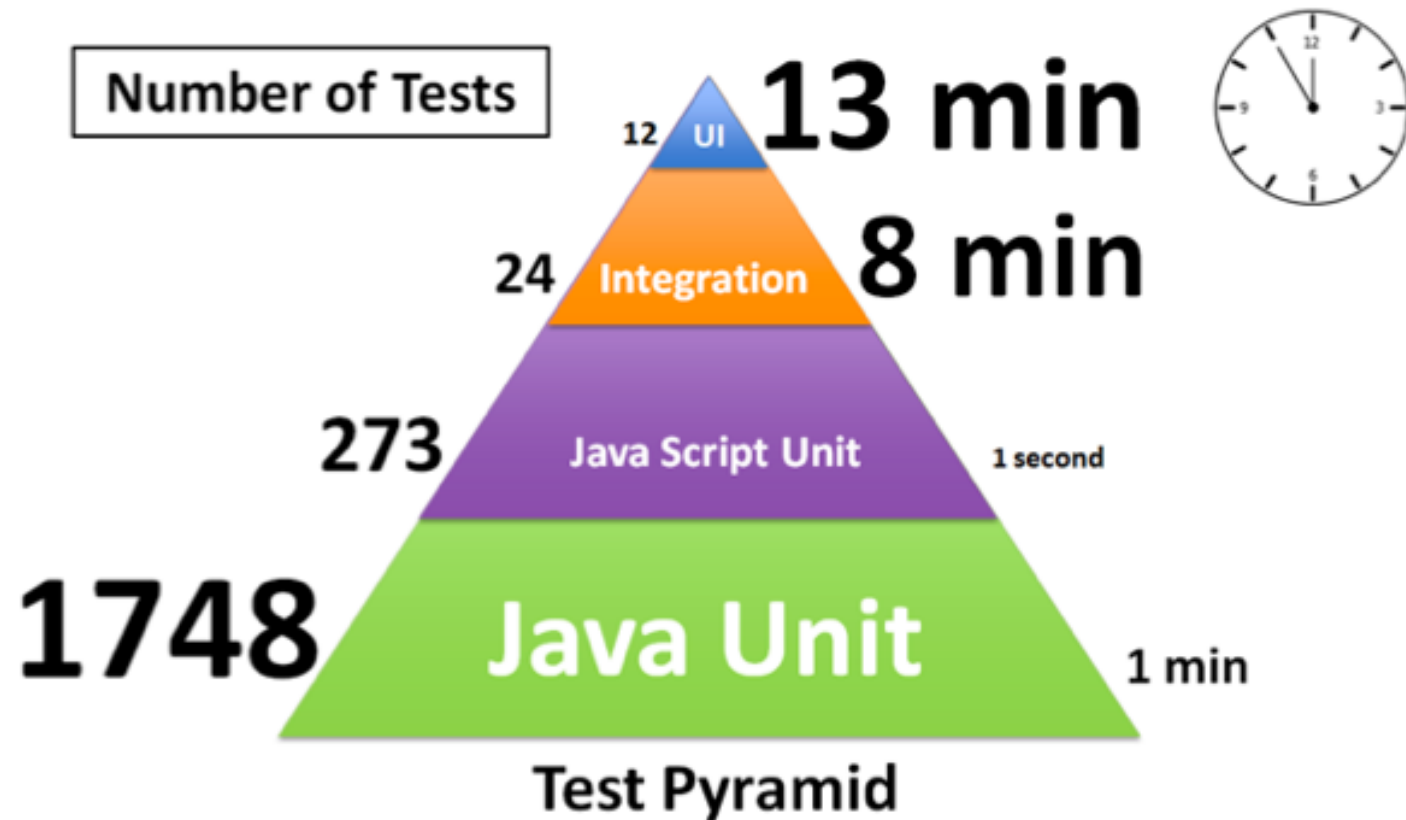
- DevOps adopts a “Shift Left” Lean alignment to increase focus on quality issues earlier in the development process.



Shift Left



Shift Left



Shift Left

- Shifting left improves quality and reduces costs.
 - Easier to automate testing when the test is done as close to the code as possible (bake quality into the product).
 - Easier for a developer to fix a defect found earlier in the dev process.

	Coding/Unit Testing	Integration	Beta Testing	Post-Release
Hours to Fix	3.2	9.7	12.2	14.8
Cost to fix (\$)	240	728	915	1,110

Data Source: (Planning Report 02-3, "The Economic Impacts of Inadequate Infrastructure for Software Testing," Prepared by RTI for National Institute of Standards and Technology, May 2002, p 7-12.)

GUI test anti-pattern

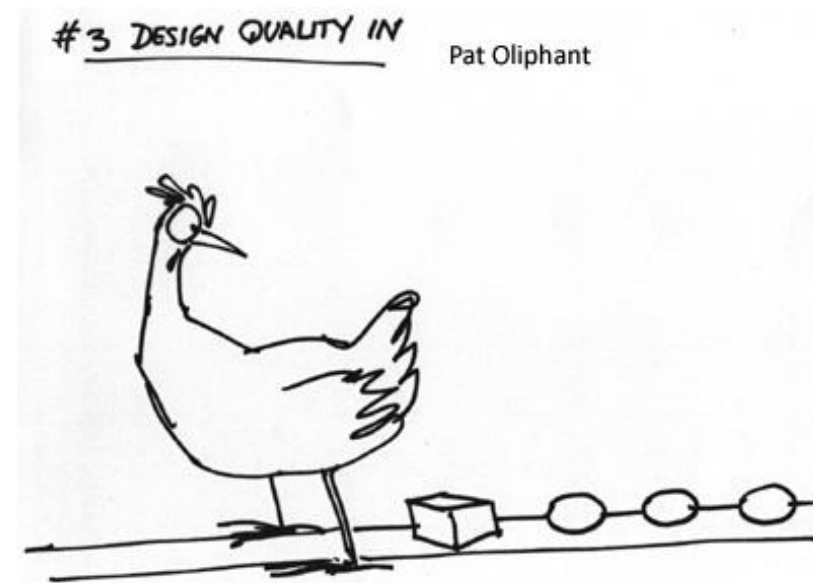
- GUI app tests use tools like Selenium to simulate web client access.
- These tests are slow, expensive to maintain, and non-deterministic.
 - Yet many companies still focus the majority of their testing effort writing these types of tests!
- Non-deterministic tests are dangerous.
 - False positives lead to alarm fatigue.
 - False negatives lead to production outages.
 - A non-deterministic test is worse than no test at all!

GUI test anti-pattern

- Modern testing practices:
 - Limit the use of automated GUI tests to key application workflows.
 - Leverage unit and API/integration tests for the majority of your automated testing.
 - Build test endpoints into your API to make it easier to test application.
 - Manufacturers build test harness interfaces into things like engines to make them easier to test during assembly.
 - Eliminate or quarantine any non-deterministic tests.
 - 100% of the tests in your test suite should pass!

The "hardening" anti-pattern

- Many companies still implement a "hardening" phase where they test code much later in the delivery cycle.
- Code is always being tested in a continuous delivery process, eliminating the need for hardening.

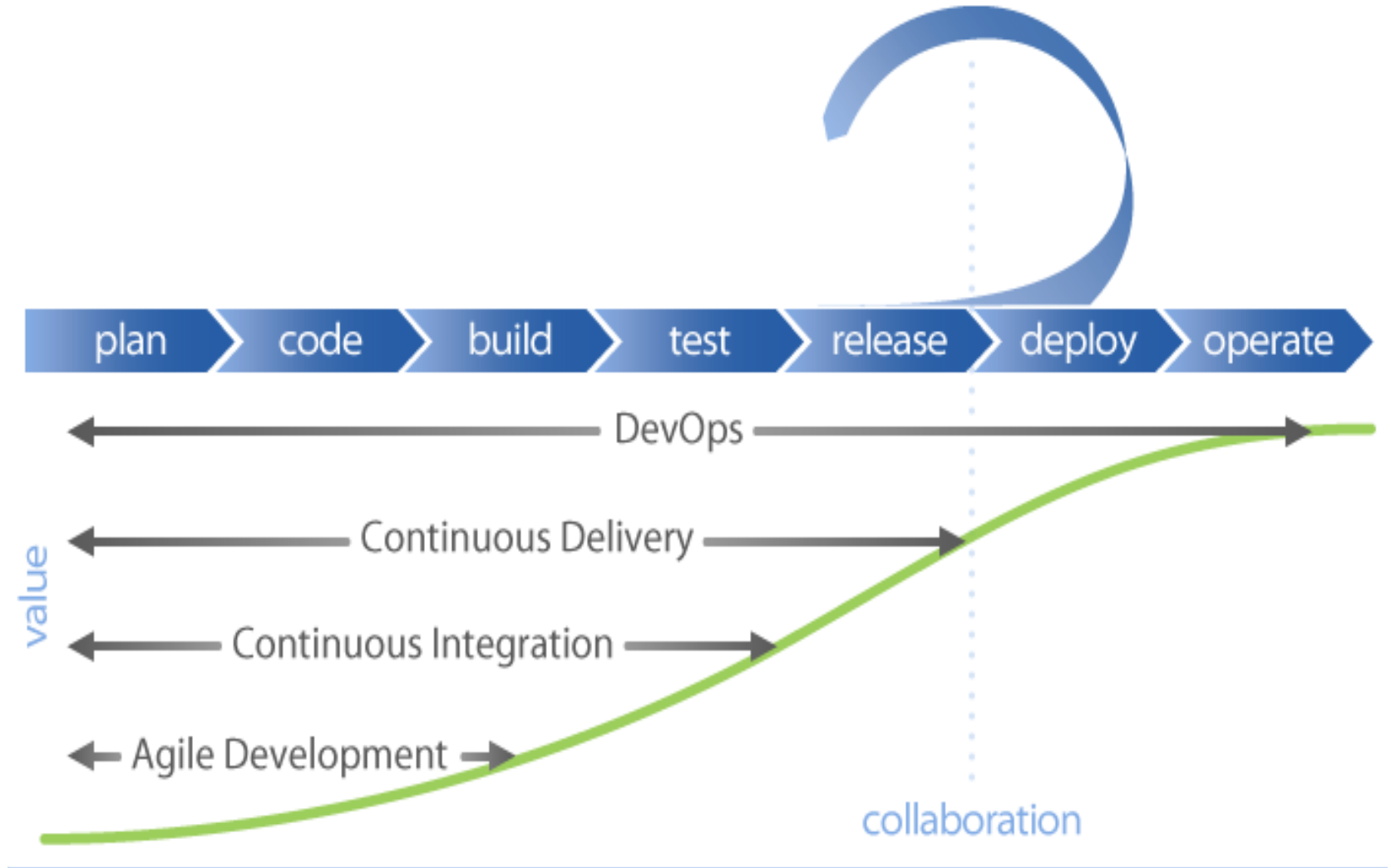


Continuous Delivery

- A **continuous delivery** process ensures that software is deployable throughout its lifecycle.
- Team prioritizes keeping the software deployable over working on new features.
- All parts of the pipeline are extensively automated including the deployment environment.



Continuous Delivery



Traditional software delivery

- Traditionally, in IT we managed the risk of introducing change by:
 - Changing infrequently and only when absolutely necessary.
 - Changing at inconvenient times when staff and vendors are at their lowest performance levels.
 - Batching changes during change windows.

Traditional software delivery

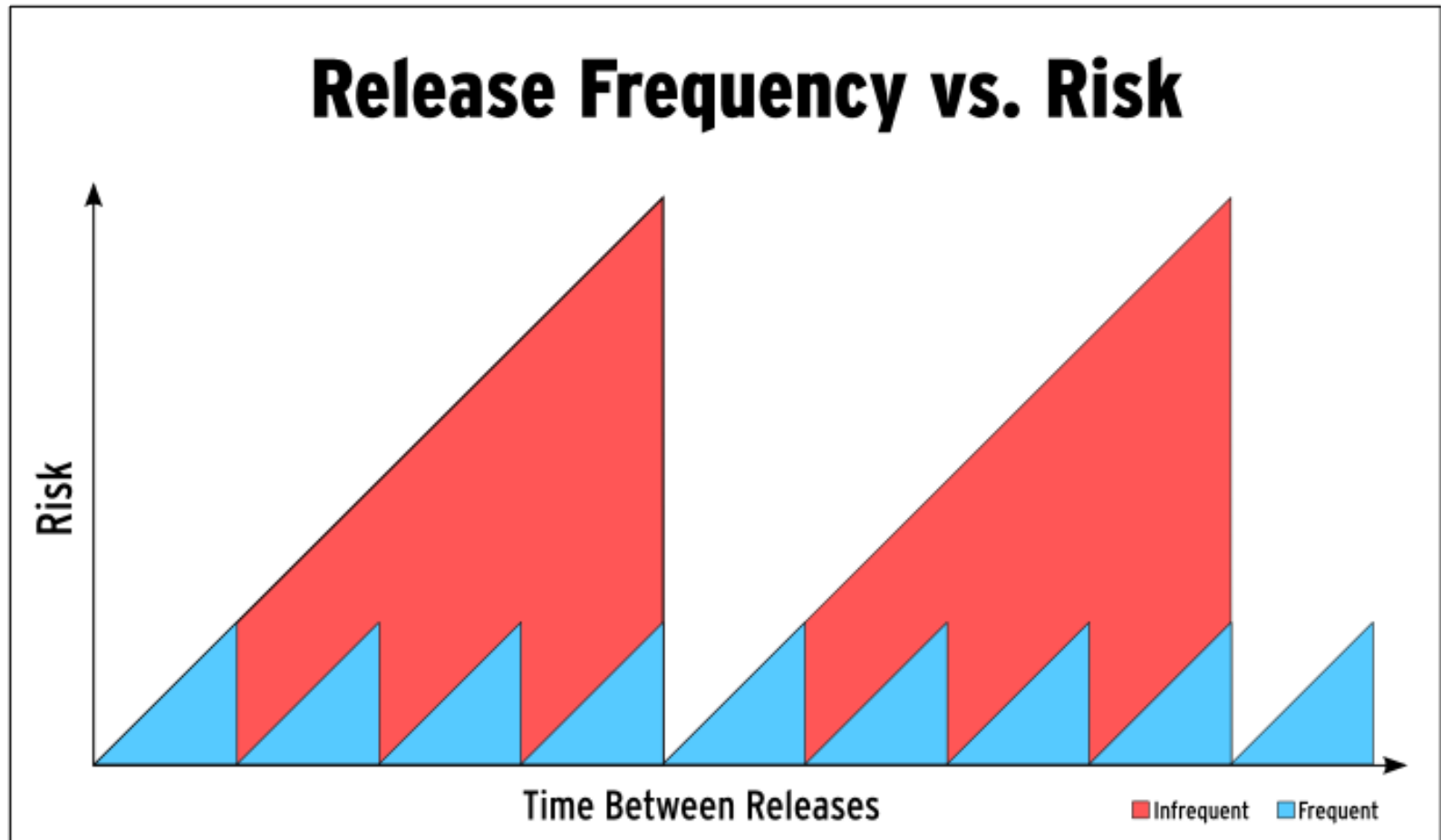
- Big-bang: everyone experiences change simultaneously
- Production testing after release. Spray and pray!
- Deployments scheduled at inconvenient hours.
- Rolling back requires another deployment. Uh, oh!



DevOps software delivery

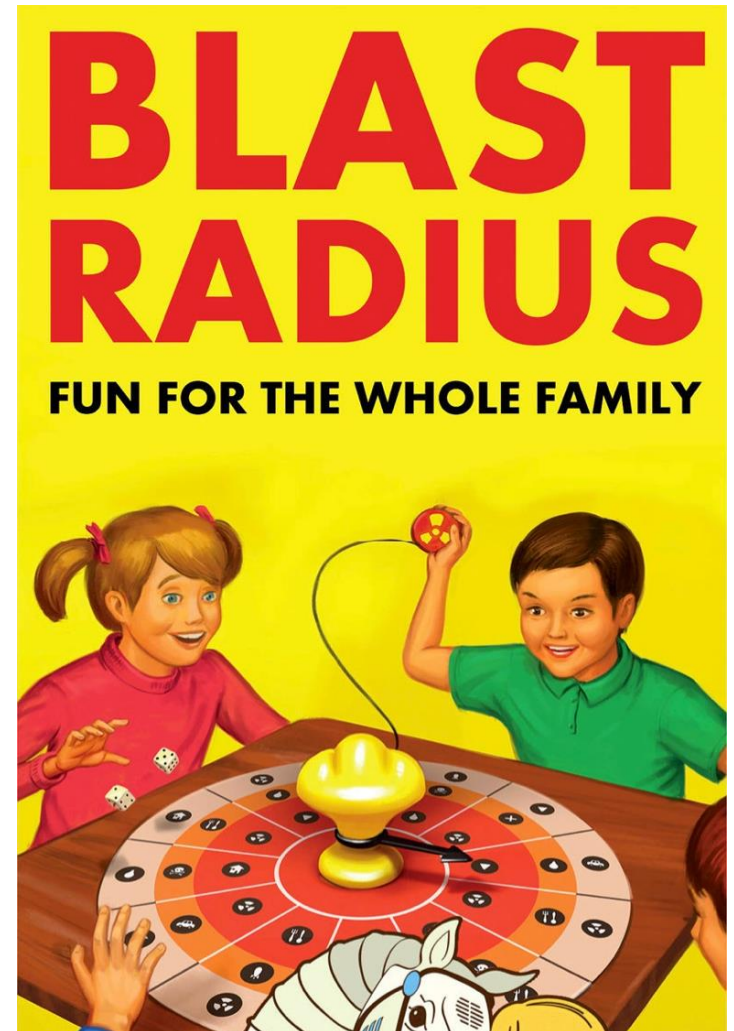
- Software changes are delivered frequently and in small batches (Little's Law!).
- Frequent software deployments allow us to deliver value faster and with lower risk.
 - Recall change failure rates for high performing organizations.

DevOps software delivery



DevOps software delivery

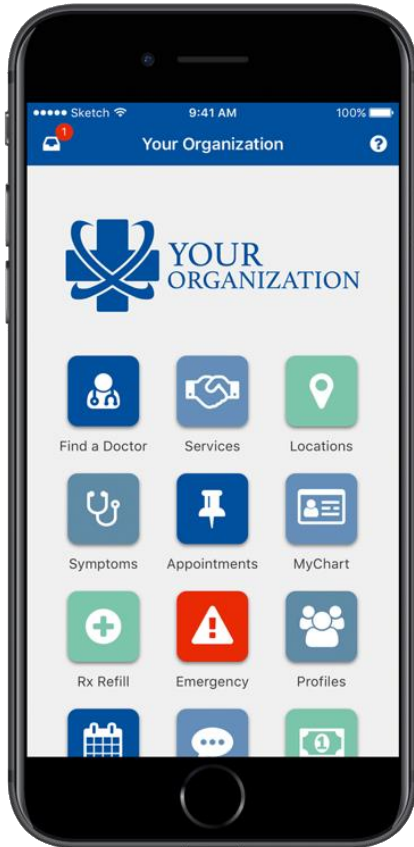
- Small changes allow us to minimize the impact or "blast radius" of change.



Deployments vs. releases

- **Deployment:** installation of new software code in an environment.
- **Release:** making new software code available to customers.
- One of the keys to successful continuous delivery is decoupling deployments from releases.

Deployments vs. releases



Dear valued customer, we are upgrading our health care platform tomorrow. You are required to upgrade your mobile app promptly at 2:35am CST tomorrow to continue to use our platform.

Deployments vs. releases

- Need to support multiple versions of services running concurrently.
 - We will always have a cutover window (unless we quiesce traffic).
- Version your APIs and API payloads.
- Leverage feature flags/toggles.

Feature Flag Pattern

- Code changes are wrapped in an if-then-else block which is controlled by a configuration variable.
 - Configuration variable allows you to enable/disable feature in different environments.
 - Work-in-progress features are deployed in production.
WHAT??
 - This process decreases software integration risks.

Infrastructure as Code

- An approach to infrastructure automation based on practices used in software development.
 - Versionable, Testable, Repeatable
- Goals
 - Infrastructure supports change and is no longer an obstacle.
 - Infrastructure change is routine.
 - Improvements are made continuously versus big bang releases.

Measurement



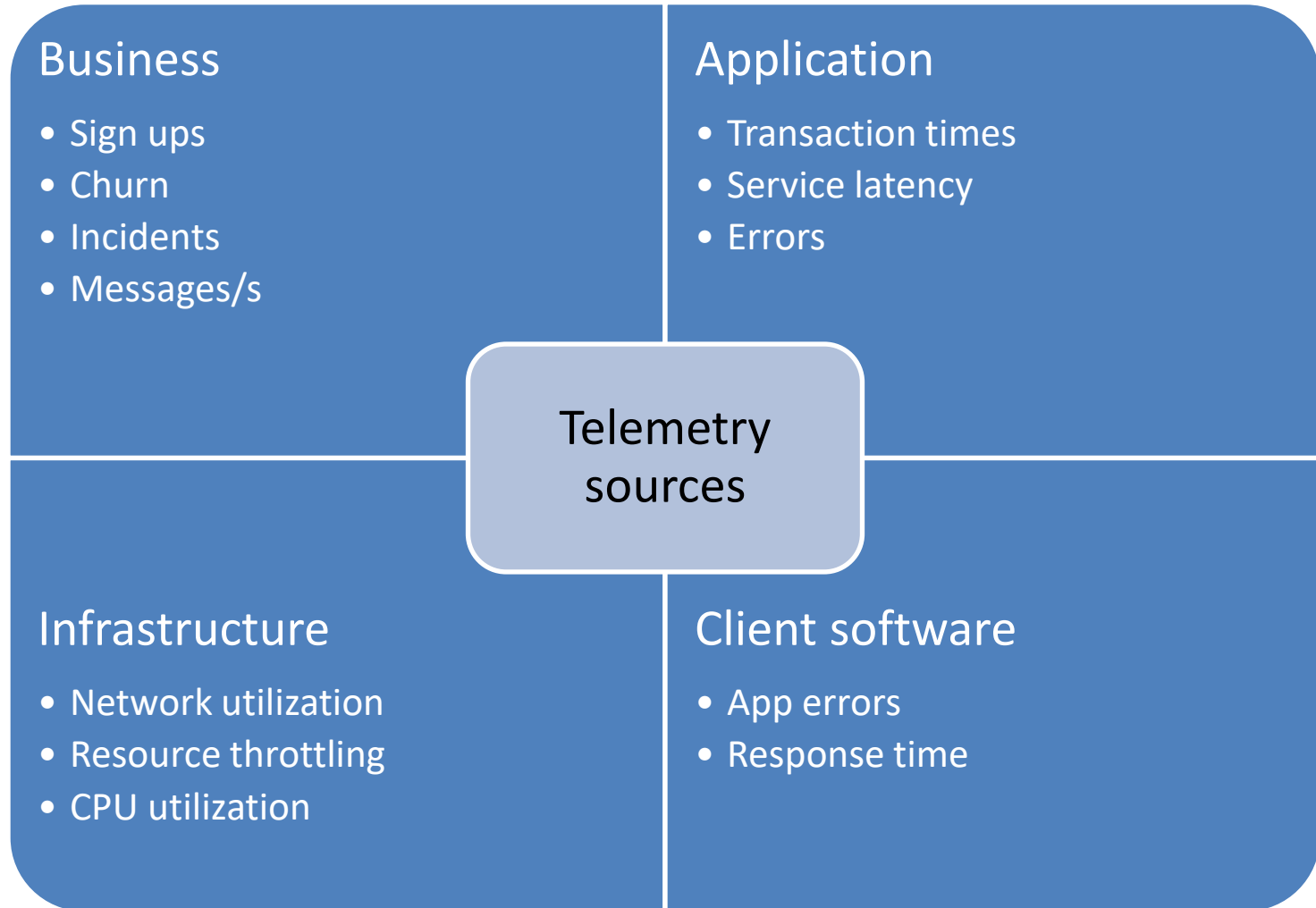
Measurement

- DevOps organizations fanatically collect data.
 - Essential feedback mechanism for improving services and validating changes.
 - Collect data from diverse resources and store in a central repository.
 - Time-series and unstructured data
 - Make it easy to search data to identify trends and correlations.

Telemetry types

- **Events:** point-in-time occurrences of significant activities.
 - Login, payment, deployment
- **Logs:** application diagnostic information and errors.
 - Website access logs, Lambda execution logs, audit logs
- **Metrics:** observable application and environment properties.
 - Memory utilization, Lambda latency, DynamoDB reads
- **Traces:** measured application execution paths.
 - Stack traces, AWS X-ray, APM

Telemetry sources



Telemetry development

Development teams determine what to measure.

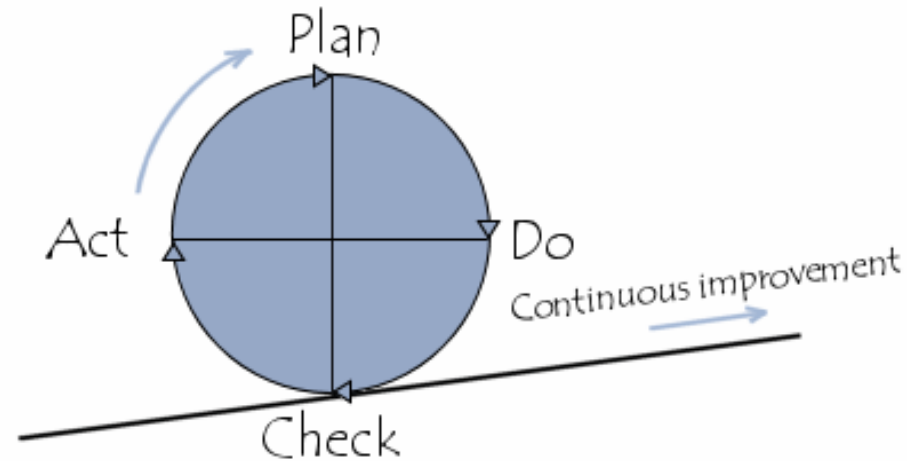
- Dashboards
- Instrumenting code
- Definition of Done = it can be measured



Telemetry development

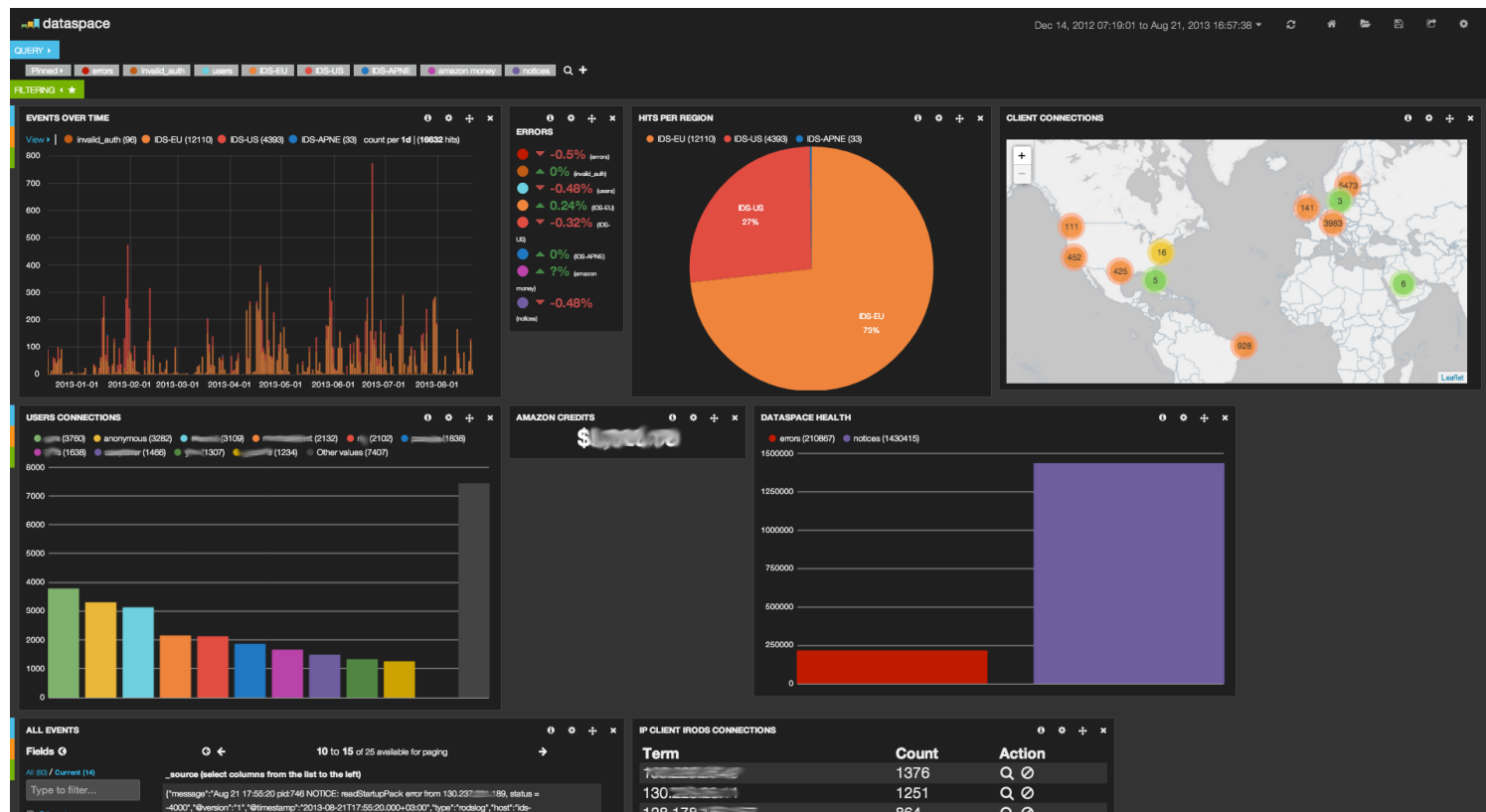
Iteratively improve telemetry:

1. Incident occurs
2. Postmortem: What telemetry could we have implemented to predict this incident?
3. Implement telemetry and add an alert.
4. Go back to step 1.



Telemetry instrumentation

- ELK stack (Elasticsearch + Logstash + Kibana)



Telemetry alarms

- Generate notifications to alert staff on service issues.
 - Weed out non-actionable notifications (log but don't email) to prevent alarm fatigue.
 - Example: failing infrastructure is not actionable if the service is self-healing.
 - Caveat: If a service is self-healing at a greater than expected rate, that might be actionable.

Measurement and decision-making

- Measurement drives decision-making in high performing DevOps organizations.
 - Sometimes our technical solutions are educated guesses.
 - “I think if we add more memory the app performance will improve.”
 - Use metrics to strengthen recommendations.
 - Any recommendation not supported by data is considered a guess and is therefore risky.
 - Cloud infrastructure gives us a cheap way to run experiments.



Parting Notes

- You cannot change a traditional IT organization into a DevOps unicorn overnight.
- Start small, prove out the effectiveness of new patterns and celebrate successes.
- Culture is the hard part.

